

생성형 AI의 특징

체크리스트

- ☑ 결정적(Deterministic) 프로그래밍과 확률적(Probabilistic) 프로그래밍의 핵심적인 사고 방식 차이를 이해했나요?
- ☑ 생성형 AI의 주요 한계점(환각, 지식 단절, 맥락 이탈)의 발생 원인을 설명할 수 있나요?
- ☑ 프롬프트 엔지니어링의 5개 핵심 구성 요소(Role, Task, Context, Format, Example)를 각각 정의할 수 있나요?
- ☑ Zero-shot과 Few-shot 프롬프팅의 차이점과 Few-shot이 필요한 상황을 인지하고 있나요?
- ☑ 일반 Instruction과 System Instruction의 역할 차이와 System Prompt의 중요성을 이해했나요?
- ☑ RTCF 프레임워크나 태그 기반 구조를 활용하여 프롬프트를 구조화(Structured Prompt)할 수 있나요?
- ☑ RAG(검색 증강 생성)의 개념과 이를 활용한 도구(GPTs, NotebookLM 등)의 동작 원리를 알고 있나요?

결정적 프로그래밍에서 확률적 프로그래밍으로

기존 프로그래밍의 특징

- 입력이 같으면 **결과도 동일**
- 규칙과 흐름이 명확함
- 예측 가능성이 높음
- 시스템 상황에 따라 정해진 반환값(오류/예외) 발생 가능

</>대표적인 예시

- 계산기 연산
- 정렬 알고리즘
- 데이터베이스 조회

오류 (Error)

메모리 부족이나 시스템 장애 등 프로그램 스스로 복구할 수 없는 치명적인 문제.

예외 (Exception)

잘못된 입력값이나 네트워크 지연 등 프로그램 실행 중 발생하지만, 코드로 미리 대비하여 복구할 수 있는 예상 밖의 상황.

i 오류와 예외는 환경에 따라 혼용되므로 엄격한 구분은 불필요합니다.

결정적 프로그래밍에서 확률적 프로그래밍으로

생성형 AI 기반 프로그래밍

- 같은 질문에도 **결과가 달라질 수 있음**
- 확률적으로 가장 그럴듯한 결과 생성
- 창의적이고 유연한 결과 가능
- 대신 **결과 통제가 어려움**

확률적 (Probabilistic)

항상 같은 결과가 아니라 확률 기반으로 결과가 결정되는 특성.

+ 장점

- 자연스러운 대화 및 다양한 표현
- 빠른 초안 생성
- 복잡한 문제 접근 가능

- 단점

- 결과 일관성 부족
- 예측 어려움
- 검증 필요

머리는 좋은데 상식이 부족하다

모델은 매우 다양한 분야 지식을 보유하고, 빠르게 초안을 생성하며 전문적인 구조를 제공합니다.

⚠ 하지만 한계가 존재합니다

- 실제 세계 규칙을 **완전히 이해하는 것은 아님**
- 사람처럼 상식과 경험을 가진 것은 아님
- 맥락 없이 **그럴듯한 답**을 생성 가능

🔑 따라서 중요한 요소

- 명확한 지시 및 충분한 맥락 제공
- 검증 절차
- 규칙과 제한 설정

하네스 (Harness)

모델이 잘못된 방향으로 동작하지 않도록 제약과 규칙을 제공하는 구조.

가드레일 (Guardrail)

모델의 위험하거나 잘못된 행동을 제한하기 위한 안전 장치.

"모른다"를 제대로 처리하지 못하는 문제 (1/3)

1. 환각 (Hallucination)

존재하지 않는 정보를 지어내는 현상입니다.

- 모델이 사실이 아닌 내용을 자신감 있게 생성
- 잘못된 링크, 논문, 코드, 통계 등을 생성 가능

🔍 발생 원인

정답을 찾는 것이 아니라

"그렇듯한 다음 토큰"을 생성하기 때문입니다.

☰ 대표적인 예시

- 없는 논문 인용
- 존재하지 않는 API 생성
- 틀린 숫자와 통계 제시

"모른다"를 제대로 처리하지 못하는 문제 (2/3)

2. 지식 단절 (Knowledge Cutoff)

모델의 학습 시점 이후에 발생한 **최신 정보가 부족한 현상**입니다.

≡ 대표적인 예시

- 최신 프레임워크 버전 모름
- 최근 사건 반영 실패
- 새로운 회사 정책 모름

🔧 해결 방식

- 웹 검색 연결 (Web Search Integration)
- 외부 데이터 연결
- **RAG** (검색 증강 생성) 기술 사용

"모른다"를 제대로 처리하지 못하는 문제 (3/3)

3. 맥락 이탈 (Context Drift)

너무 많은 정보로 인해 핵심 목적을 잃고 원래 질문과 **관계없는 방향으로 흐름이 이동**하는 현상입니다. (흔히 "삼천포 간다"고 표현)

예시

- 핵심 요구사항 누락
- 관련 없는 기능 추가
- 원래 목적과 다른 결과 생성

발생 원인

- **긴 대화/세션**
 - 대화가 길어지면 토큰 효율을 위해 대화를 압축하며, 이 과정에서 손실 발생 가능
- **과도한 검색**
- **불필요한 문맥 혼합**
 - AI의 '메모리' 기능으로 이전 대화 내용이 섞여 들어옴
- **애매한 지시사항**

생성형 AI를 잘 사용하는 방법

프롬프트 엔지니어링 (Prompt Engineering)

모델이 이해하기 쉬운 형태로 요청을 작성하고, 환각과 맥락 이탈을 완화하기 위한 입력 설계 기술입니다.

5가지 핵심 요소

- **역할 (Role)**: AI가 담당할 특정 페르소나나 직업 부여 (예: 시니어 개발자)
- **목적 (Objective)**: 달성하고자 하는 구체적인 작업 목표
- **제약조건 (Constraint)**: 글자 수, 금지어, 필수 포함 내용 등 규칙
- **출력 형식 (Output Format)**: 마크다운, JSON, 표 등 결과물 형태
- **예시 (Example)**: 결과물의 형태를 보여주는 실제 샘플 (Few-shot)

프롬프트 (Prompt)

모델에게 전달하는 입력과 지시사항.



편의상 **Gemini** 환경에서 실습을 진행합니다.
(구글 계정 필요)

Zero-shot과 Few-shot (1/2)

Zero-shot

예시 없이 바로 요청을 전달하며, 모델의 일반적인 학습 능력을 그대로 활용합니다.

예시

- "이 문장을 요약해줘"
- "다음 문장의 감정을 분석해줘: '정말 최고의 영화였어요!'"

Few-shot

원하는 결과물의 예시를 함께 제공하여, 결과의 스타일과 품질을 명확히 유도합니다.

예시

- 입력/출력 예시를 2~3개 제공
- 원하는 형식을 학습시키듯 전달

Zero-shot과 Few-shot (1/2)

구분	입력 (Prompt)	출력 (Output)
Zero-shot	"다음 문장의 감정을 분석해줘: '정말 최고의 영화였어요!'"	"이 문장은 긍정적인 감정을 나타냅니다." (모델 기본 말투)
Few-shot	"리뷰: 별로예요 -> 감정: 부정 리뷰: 괜찮네요 -> 감정: 중립 리뷰: 정말 최고의 영화였어요! -> 감정:"	"긍정" (제공된 예시의 짧은 형식 준수)

실습: Zero-shot과 Few-shot

기본 실습

1. 간단한 요청(Zero-shot)을 입력하고 결과를 확인합니다.
2. 해당 요청에 ()을 few shot 형식으로 바꿔주세요를 추가하여 개선된 결과를 확인합니다.

이미지 프롬프트 맞추기 실습

- 생성형 AI 이미지를 보고 **50자 이내 한글** 프롬프트를 유추합니다.
(고유명사는 영어)
- 프롬프트와 생성한 이미지를 리더보드에 제출합니다. (최소 1회)

 참고: **Arena Image AI** 활용

(구글 소셜 로그인 필요)

- Direct > Max 클릭 후 테스트 이미지 선택
- Side by Side 모드로 2개 이상 비교 가능
- 대화상자에 프롬프트 입력 후 엔터
- 결과를 확인하여 본래 이미지와 비교

 **팁: LLM의 이미지 해석 기능을 활용해 보세요.**

Instruction과 System Instruction

일반 Instruction

사용자의 **요청 자체**이며, 현재 작업에 대한 직접적인 지시입니다.

예시

- "이 기사의 핵심 내용을 3줄로 요약해줘"
- "다음 파이썬 코드를 자바스크립트로 변환해줘"
- "React의 useEffect를 초보자에게 설명해줘"

System Instruction

모델 **전체의 행동 규칙**을 정의하며, 서비스 운영에서 매우 중요합니다.

예시

- "절대 개인정보를 출력하지 마라"
- "한국어로만 답변하라"
- "코드 블록만 출력하라"

System Prompt

모델 전체 동작 방식을 정의하는 최상위 지시사항.

System Instruction 적용

System Instruction 실습

1. Gemini 좌측 하단 설정 및 도움말 클릭
2. 개인별 컨텍스트 > Gemini 요청 사항 > 추가 이동
3. System 지시사항을 입력합니다.
4. 지시 반영 **전후의 결과를 비교**해 봅니다.

프롬프트 프레임워크 (1/2) - RTCF

사용자의 Instruction을 체계적으로 반영할 수 있게 구성요소를 정의하여 작성하는 체계입니다.

- ☑ **Role(역할):** AI가 어떠한 페르소나나 전문가로서 답변할 것인지 지정
- ☑ **Task(작업):** AI가 수행해야 할 구체적인 목표 및 핵심 지시사항 명시
- ☑ **Context(맥락):** 작업 수행에 필요한 배경 지식, 타겟 대상, 상황 정보 등 제공
- ☑ **Format(형식):** 최종 결과물의 구조, 길이, 출력 형태 (마크다운, 표, JSON 등) 지정

* Tip: Role 부여 시 연차(예: 10년 차) 명시와 실제 성능 향상 논란 존재. '특정 분야의 전문가(예: 성능 최적화 전문가)' 등 구체적 역량 지정이 더 권장됨.

적용 예시

- **Role:** 당신은 친절한 시니어 프론트엔드 개발자 멘토입니다.
- **Task:** React의 useState 개념을 설명해 주세요.
- **Context:** 코딩을 처음 배우는 주니어가 대상입니다. 이해하기 쉽게 일상생활의 비유를 들어주세요.
- **Format:** 제목, 개념 설명, 비유를 활용한 예시, 짧은 코드 스니펫 순서의 마크다운으로 작성해 주세요.

프롬프트 프레임워크 (2/2) - 태그 기반 구조

XML 스타일 태그 활용

역할, 작업명, 문맥 등을 태그로 **명확히 분리**하여 모델이 정보를 혼동하지 않게 합니다.

태그 (Tag): 데이터의 시작과 끝을 알리고 의미를 부여 (예: 내용)

XML: 사용자가 직접 태그를 정의하여 데이터를 구조적으로 표현하는 마크업 언어

★언제 유용할까?

- 프롬프트가 길어질 때
- 여러 개의 참고 문서(Context)를 함께 전달할 때
- **일관된 답변**이 중요한 AI 서비스나 에이전트를 구축할 때

Structured Prompt

구조화된 형태로 정보를 구분하여 전달하는 프롬프트 방식.

RAG (Retrieval-Augmented Generation, 검색 증강 생성)

RAG란 무엇인가요?

모델 내부 지식만 사용하는 것이 아니라, **외부 문서를 검색하여 함께 답변을 생성**하는 기술입니다.



1. 질문 입력



2. 관련 문서 검색



3. 모델에 결과 제공



4. 답변 생성

✓ RAG의 장점

- 최신 정보 반영 가능
- 회사 내부 문서 활용 가능
- **환각 현상 감소** 및 정확도 향상

Retrieval (검색)

필요한 정보를 외부 저장소에서 검색하는 과정.

RAG 활용 도구

GPTs / Gems

특정 목적에 맞게 **커스터마이징된 AI**입니다. 지시사항과 문서를 함께 구성할 수 있습니다.

대표 예시

- 회사 FAQ 챗봇
- 면접 준비 도우미
- 코드 리뷰 봇

Project / NotebookLM

프로젝트 단위로 문맥과 파일을 관리하거나, 사용자가 **업로드한 문서**를 기반으로 답변을 생성합니다.

NotebookLM

Google의 문서 기반 AI 연구/학습 도구.

사용자가 업로드한 자료를 기반으로
요약, 질의응답, 학습 보조를 수행합니다.

RAG 실습 해보기

Gems 실습

1. Gemini에서 Gem 관리자 > 내 Gems > 새 Gem 이동
2. 요청사항과 지식에 필요한 내용을 입력합니다.
(요청사항은 Gemini 기능으로 개선 가능합니다.)
3. 우측 저장 후 생성된 Gem을 공유해 봅니다.

NotebookLM 실습

1. NotebookLM 우측 상단 새로 만들기 > 소스 추가에서 문서를 가져옵니다.
2. 채팅 후 유용한 결과를 메모에 저장합니다.
3. 저장된 메모를 더보기에서 소스로 변환할 수 있습니다.
4. 스튜디오에서 팟캐스트나 슬라이드를 생성해 봅니다.

* Gemini 환경에서도 Notebook을 확인할 수 있습니다.